

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2
По дисциплине: «КиСБ»
Тема: «Избыточное кодирование данных в информационных системах. Код
Хемминга. Итеративные коды»

Выполнил:
Студент 4-го курса
Группы АС-66
Рогожин А. С.
Проверил:
Хацкевич А.С.

Брест 2026

Цель работы: приобретение практических навыков кодирования/декодирования двоичных данных при использовании кода Хемминга. приобретение практических навыков кодирования/декодирования двоичных данных при использовании итеративных кодов .

Ход работы

Вариант 20

Часть 1

На основе информационного сообщения, представленного символами русского/английского алфавитов, служебными символами и цифрами, содержащегося в некотором текстовом файле (согла- 42 Лабораторная работа N2 4 совать с преподавателем), сформировать информационное сообщение в двоичном виде; длина сообщения в бинарном виде должна быть не менее 16 символов. Для выполнения этого задания можно использовать коды ASCII символов алфавита либо результаты лабораторной —ты N2 3. 2. Для полученного информационного слова проверить матрицу Хемминга (значение минимального кодового расстояния согласовать с преподавателем). 3. Используя построенную матрицу, вычислить избыточные символы (слово X). 4. Принять слово следующим числом ошибок: 0, 1, 2. Поскольку ошибки определяется (генерируется) случайным образом. 5. Для полученного слова $Y_p = U_k, Y_r$, используя уже известную проверочную матрицу Хемминга, вновь вычислить избыточные символы (обозначим их Г), используя выражение (4.6). 6. Вычислить и проанализировать синдром. В случае, если анализ синдрома показал, что было передано с ошибкой (или 2 ошибками), унарный вектор ошибки $E_{,,} = \psi, \phi$, и исправить одиночную ошибку, используя формулу (4.7); проанализировать ситуацию при возникновении ошибки в 2 битах. 7. Результаты оформить в виде отчета по установленным правилам. Файл анализируемой информации (а скриншоты и интерфейс приложения) должен содержать исходное информационное сообщение, значения величин K, r, n, проверочную матрицу Хемминга слова X, X, Y_p, Y_r, Y_r' , синдром S, вектор ошибки Eп. Программа не должна быть чувствительна к длине информационного сообщения.

Часть 2

Разработать собственное приложение, которое позволяет выполнять следующие операции: 1) вписывать произвольное двоичное представление информационного слова X_k (кодируемой информации) длиной K битов в двумерную матрицу размерностью в соответствии с вариантом либо в трехмерную матрицу в соответствии с вариантом (указаны в табл. 5.2); 2) вычислять проверочные биты (биты паритетов): а) по двум; б) по трем; в) по четырем направлениям (группам паритетов); 3) формировать кодовое слово X, присоединением избыточных символов к информационному слову; 4) генерировать ошибку произвольной кратности ($i, i > 0$), распределенную случайным (фазом среди символов слова X, в результате чего формируется кодовое слово Y_p ; 5) определять местоположение ошибочных символов итеративным кодом в слове Y_p в соответствии с используемыми группами паритетов по пункту (2) и исправлять ошибочные символы (результат исправления слово Y_p'); б) выполнять анализ корректирующей способности используемого кода (количественная оценка) путем сравнения соответствующих слов X, и $Y_{,,}'$; результат анализа может быть представлен в виде отношения общего числа сгенерированных кодовых слов с ошибками

определенной одинаковой кратности (с одной ошибкой, с двумя ошибками и т. д.) к числу кодовых слов, содержащих ИзбыточнЕ данных в инфрмационных системах. Итеративные коды 55 ошибки этой кратности, которые правильно обнаружены и которые правильно скорректированы.

```
import tkinter as tk
from tkinter import ttk, messagebox, scrolledtext, filedialog
import random

class IterativeHammingVar5App:
    def __init__(self, root):
        self.root = root
        self.root.title("Лабораторная работа: Итеративные коды (Вариант 5)")
        self.root.geometry("1100x750")

        self.style = ttk.Style()
        self.style.theme_use('clam')

        self.create_widgets()

    def create_widgets(self):
        left_frame = ttk.LabelFrame(self.root, text="Параметры и ввод данных (k = 40)")
        left_frame.pack(side=tk.LEFT, fill=tk.Y, padx=10, pady=10, ipadx=5, ipady=5)

        right_frame = ttk.LabelFrame(self.root, text="Результаты и отчет")
        right_frame.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True, padx=10, pady=10)

        # Структура матрицы
        ttk.Label(left_frame, text="Структура матрицы:").grid(row=0, column=0,
            sticky=tk.W, pady=5)
        self.matrix_type_var = tk.StringVar(value="2D: 5 x 8")
        matrix_combo = ttk.Combobox(left_frame, textvariable=self.matrix_type_var,
            state="readonly", width=22)
        matrix_combo['values'] = ("2D: 5 x 8", "2D: 4 x 10", "3D: 5 x 4 x 2", "3D: 2 x
            10 x 2")
        matrix_combo.grid(row=0, column=1, pady=5)

        # Группы паритетов
        ttk.Label(left_frame, text="Группы паритетов:").grid(row=1, column=0,
            sticky=tk.W, pady=5)
        self.parity_groups_var = tk.StringVar(value="2 и 3 группы")
        parity_combo = ttk.Combobox(left_frame, textvariable=self.parity_groups_var,
            state="readonly", width=22)
        parity_combo['values'] = ("2 и 3 группы", "2, 3, 4 и 5 групп")
        parity_combo.grid(row=1, column=1, pady=5)

        # Загрузка из txt
        ttk.Label(left_frame, text="Источник данных:").grid(row=2, column=0,
            sticky=tk.W, pady=5)
        btn_load_txt = ttk.Button(left_frame, text="Загрузить из .txt файла",
            command=self.load_from_txt)
        btn_load_txt.grid(row=2, column=1, sticky=tk.EW, pady=5)

        ttk.Label(left_frame, text="Информационное слово (40 бит):").grid(row=3,
            column=0, columnspan=2, sticky=tk.W, pady=(5, 0))

        self.info_text_entry = tk.Text(left_frame, height=4, width=32, font=("Courier",
            10))
```

```

self.info_text_entry.grid(row=4, column=0, columnspan=2, pady=5)
self.generate_random_bits()

btn_rand = ttk.Button(left_frame, text="Случайные биты",
command=self.generate_random_bits)
btn_rand.grid(row=5, column=0, sticky=tk.W, pady=5)

ttk.Label(left_frame, text="Кратность ошибки (i):").grid(row=6, column=0,
sticky=tk.W, pady=5)
self.error_count_var = tk.IntVar(value=1)
err_spin = ttk.Spinbox(left_frame, from_=0, to=10,
textvariable=self.error_count_var, width=5)
err_spin.grid(row=6, column=1, sticky=tk.W, pady=5)

btn_run = ttk.Button(left_frame, text="Запустить моделирование",
command=self.run_simulation)
btn_run.grid(row=7, column=0, columnspan=2, pady=15, sticky=tk.EW)

# Правая панель с результатами
self.result_box = scrolledtext.ScrolledText(right_frame, wrap=tk.WORD,
font=("Courier", 10))
self.result_box.pack(fill=tk.BOTH, expand=True, padx=5, pady=5)

def generate_random_bits(self):
bits = "".join(str(random.randint(0, 1)) for _ in range(40))
self.info_text_entry.delete("1.0", tk.END)
self.info_text_entry.insert("1.0", bits)

def load_from_txt(self):
file_path = filedialog.askopenfilename(
title="Выберите текстовый файл (.txt)",
filetypes=[("Text files", "*.txt"), ("All files", "*.*")]
)
if not file_path:
return

try:
with open(file_path, 'r', encoding='utf-8') as f:
text_content = f.read()
except Exception as e:
messagebox.showerror("Ошибка", f"Не удалось прочитать файл:\n{e}")
return

binary_chars = []
for char in text_content:
b_val = ord(char)
b_str = format(b_val & 0xFF, '08b')
binary_chars.append(b_str)

full_binary = "".join(binary_chars)

if len(full_binary) >= 40:
bits_40 = full_binary[:40]
else:
bits_40 = (full_binary * (40 // len(full_binary) + 1))[:40]

self.info_text_entry.delete("1.0", tk.END)
self.info_text_entry.insert("1.0", bits_40)

```

```

messagebox.showinfo(
    "Успешно",
    f"Файл прочитан!\nСформировано ровно 40 бит из содержимого файла."
)

def get_matrix_dims(self):
    m_type = self.matrix_type_var.get()
    if "5 x 8" in m_type:
        return (5, 8, None)
    elif "4 x 10" in m_type:
        return (4, 10, None)
    elif "5 x 4 x 2" in m_type:
        return (5, 4, 2)
    elif "2 x 10 x 2" in m_type:
        return (2, 10, 2)
    return (5, 8, None)

def parse_bits(self, text):
    clean = "".join([c for c in text if c in ('0', '1')])
    if len(clean) != 40:
        return None
    return [int(c) for c in clean]

def compute_parities(self, data_bits, dims, num_groups):
    k1, k2, z = dims
    parities = {}

    if z is None:
        matrix = [data_bits[i*k2:(i+1)*k2] for i in range(k1)]

        if "2" in num_groups or "2" in self.parity_groups_var.get():
            row_parities = [sum(row) % 2 for row in matrix]
            parities['row'] = row_parities

        if "3" in num_groups or "3" in self.parity_groups_var.get():
            col_parities = [sum(matrix[r][c] for r in range(k1)) % 2 for c in
range(k2)]
            parities['col'] = col_parities

        if "4" in self.parity_groups_var.get():
            diag_parities = []
            for d in range(-(k1-1), k2):
                diag = [matrix[r][c] for r in range(k1) for c in range(k2) if c - r
== d]

                if diag:
                    diag_parities.append(sum(diag) % 2)
            parities['diag'] = diag_parities

        if "5" in self.parity_groups_var.get():
            anti_parities = []
            for d in range(k1 + k2 - 1):
                anti = [matrix[r][c] for r in range(k1) for c in range(k2) if r + c
== d]

                if anti:
                    anti_parities.append(sum(anti) % 2)
            parities['anti'] = anti_parities
    else:
        matrix_3d = [[[data_bits[r*k2*z + c*z + layer] for layer in range(z)] for c
in range(k2)] for r in range(k1)]

```

```

        if "2" in self.parity_groups_var.get():
            parities['layer'] = [sum(matrix_3d[r][c]) % 2 for r in range(k1) for c
in range(k2)]
        if "3" in self.parity_groups_var.get():
            parities['row'] = [sum(matrix_3d[r][c][l] for c in range(k2) for l in
range(z)) % 2 for r in range(k1)]
        if "4" in self.parity_groups_var.get():
            parities['col'] = [sum(matrix_3d[r][c][l] for r in range(k1) for l in
range(z)) % 2 for c in range(k2)]
        if "5" in self.parity_groups_var.get():
            parities['depth'] = [sum(matrix_3d[r][c][l] for r in range(k1) for c in
range(k2)) % 2 for l in range(z)]

    return parities

def run_simulation(self):
    raw_text = self.info_text_entry.get("1.0", tk.END)
    data_bits = self.parse_bits(raw_text)

    if data_bits is None:
        messagebox.showerror("Ошибка", "Информационное слово должно содержать ровно
40 двоичных символов (0 и 1).")
        return

    dims = self.get_matrix_dims()
    groups_mode = self.parity_groups_var.get()
    num_groups_str = "2 и 3" if "2 и 3" in groups_mode else "2, 3, 4 и 5"

    # 1. Вычисление исходных паритетов
    orig_parities = self.compute_parities(data_bits, dims, num_groups_str)

    flat_parities = []
    for k_val in orig_parities.values():
        flat_parities.extend(k_val)
    X_n = data_bits + flat_parities

    # 2. Внесение ошибок
    i_err = self.error_count_var.get()
    if i_err > len(X_n):
        i_err = len(X_n)

    error_indices = random.sample(range(len(X_n)), i_err)
    Y_n = list(X_n)
    for idx in error_indices:
        Y_n[idx] ^= 1

    # 3. Прием и пересчет паритетов
    y_data = Y_n[:40]
    new_parities = self.compute_parities(y_data, dims, num_groups_str)

    # Вычисление синдромов
    syndromes = {}
    error_detected = False
    for key in orig_parities:
        syn = [o ^ n for o, n in zip(orig_parities[key], new_parities[key])]
        syndromes[key] = syn
        if sum(syn) > 0:
            error_detected = True

```

```

# Исправление ошибок
Y_n_corrected = list(Y_n)
if error_detected:
    k1, k2, z = dims
    if z is None:
        r_syn = syndromes.get('row', [])
        c_syn = syndromes.get('col', [])
        for r in range(k1):
            for c in range(k2):
                flat_idx = r * k2 + c
                if r < len(r_syn) and c < len(c_syn) and r_syn[r] == 1 and
c_syn[c] == 1:
                    Y_n_corrected[flat_idx] ^= 1
            else:
                l_syn = syndromes.get('layer', [])
                r_syn = syndromes.get('row', [])
                c_syn = syndromes.get('col', [])
                d_syn = syndromes.get('depth', [])

                for r in range(k1):
                    for c in range(k2):
                        for layer in range(z):
                            flat_idx = r * k2 * z + c * z + layer
                            match_l = (not l_syn) or (r * k2 + c < len(l_syn) and l_syn[r
* k2 + c] == 1)

                            match_r = (not r_syn) or (r < len(r_syn) and r_syn[r] == 1)
                            match_c = (not c_syn) or (c < len(c_syn) and c_syn[c] == 1)
                            match_d = (not d_syn) or (layer < len(d_syn) and
d_syn[layer] == 1)

                            if match_l and match_r and match_c and match_d and (l_syn
or r_syn or c_syn or d_syn):
                                Y_n_corrected[flat_idx] ^= 1

corrected_data_bits = Y_n_corrected[:40]

# Точное соответствие проверки (Сравнение исходных информационных бит с
восстановленными)
match_status = "ДА" if (data_bits == corrected_data_bits) else "НЕТ"

# Формирование отчета в исходном стиле
report = []
report.append("="*65)
report.append(" ОТЧЕТ ПО МОДЕЛИРОВАНИЮ ИТЕРАТИВНОГО КОДИРОВАНИЯ (ВАРИАНТ 5) ")
report.append("="*65)
report.append(f"Длина информационного слова (k): 40 бит")
report.append(f"Выбранная структура матрицы: {self.matrix_type_var.get()}")
report.append(f"Используемые группы паритетов: {groups_mode}")
report.append("-" * 65)
report.append(f"Информационное слово X_k:\n {''.join(map(str, data_bits))}")
report.append(f"\nИзбыточные символы X_r (по группам):")
for k, v in orig_paritys.items():
    report.append(f" [{k}]: {''.join(map(str, v))}")
report.append(f"\nПолное кодовое слово X_n (длина {len(X_n)}):\n
{''.join(map(str, X_n))}")
report.append("-" * 65)
report.append(f"Заданная кратность ошибки (i): {i_err}")
report.append(f"Позиции внесенных ошибок (0-indexed): {error_indices}")

```

Результат:

